

Lab Sheet 6 (Full Stack)

PRIYANSHU SHARMA CU240250980 (B) (47)

Q1. Server Setup & Building the REST API .

Q1(a) Project Setup

```
api-lab/  
  server/  
    server.js  
    package.json  
    .env  
    .gitignore  
  client/  
    index.html  
    script.js  
  client-react/  
    src/  
  
# Create project folders  
mkdir api-lab  
cd api-lab  
mkdir server client client-react  
# Go to server  
cd server  
npm init -y  
npm install express cors dotenv  
npm install --save-dev nodemon  
  
.env  
PORT=5000  
API_KEY=mysecretkey  
  
# .gitignore  
node_modules/  
.env  
  
// package.json - scripts section  
"scripts": {  
  "start": "node server.js",  
  "dev": "nodemon server.js"  
}  
  
C:\api-lab\server> npm run dev  
  
> server@1.0.0 dev  
> nodemon server.js  
  
[nodemon] starting `node server.js`  
Server running at http://localhost:5000
```

Q1(b) Complete Express REST API – server/server.js

```
const express = require("express");  
const cors = require("cors");  
require("dotenv").config();  
const app = express();  
const PORT = process.env.PORT || 5000;  
const API_KEY = process.env.API_KEY || "mysecretkey";  
// Middleware
```

```

app.use(cors());
app.use(express.json());
// Custom logger middleware
app.use((req, res, next) => {
  console.log(
    `[${new Date().toISOString()}] ${req.method} ${req.originalUrl}`
  );
  next();
});
// Authentication middleware
function authCheck(req, res, next) {
  const key = req.headers["x-api-key"];
  if (!key || key !== API_KEY) {
    return res.status(401).json({
      success: false,
      error: "Unauthorized: valid x-api-key is required"
    });
  }
  next();
}
// In-memory task data
let tasks = [
  { id: 1, title: "Learn Node.js", completed: false },
  { id: 2, title: "Learn Express.js", completed: false },
  { id: 3, title: "Build REST API", completed: false },
  { id: 4, title: "Learn React", completed: false },
  { id: 5, title: "Test API with Postman", completed: false }
];
// GET all tasks
app.get("/api/tasks", (req, res) => {
  res.json({
    success: true,
    data: tasks
  });
});
// GET one task
app.get("/api/tasks/:id", (req, res) => {
  const id = Number(req.params.id);
  const task = tasks.find(t => t.id === id);
  if (!task) {
    return res.status(404).json({
      success: false,
      error: "Task not found"
    });
  }
  res.json({
    success: true,
    data: task
  });
});
// POST task
app.post("/api/tasks", authCheck, (req, res) => {
  const { title, completed = false } = req.body;
  if (!title || typeof title !== "string") {
    return res.status(400).json({
      success: false,
      error: "Title is required"
    });
  }
  const newTask = {
    id: tasks.length
      ? Math.max(...tasks.map(t => t.id)) + 1
      : 1,
    title: title.trim(),
    completed: Boolean(completed)
  };

```

```

    };
    tasks.push(newTask);
    res.status(201).json({
      success: true,
      message: "Task created successfully",
      data: newTask
    });
  });
  // PUT task
  app.put("/api/tasks/:id", authCheck, (req, res) => {
    const id = Number(req.params.id);
    const task = tasks.find(t => t.id === id);
    if (!task) {
      return res.status(404).json({
        success: false,
        error: "Task not found"
      });
    }
    const { title, completed } = req.body;
    if (title !== undefined) task.title = String(title);
    if (completed !== undefined) task.completed = Boolean(completed);
    res.json({
      success: true,
      message: "Task updated successfully",
      data: task
    });
  });
  // DELETE task
  app.delete("/api/tasks/:id", authCheck, (req, res) => {
    const id = Number(req.params.id);
    const index = tasks.findIndex(t => t.id === id);
    if (index === -1) {
      return res.status(404).json({
        success: false,
        error: "Task not found"
      });
    }
    const deletedTask = tasks.splice(index, 1)[0];
    res.json({
      success: true,
      message: "Task deleted successfully",
      data: deletedTask
    });
  });
  // 404 handler for unmatched URLs
  app.use((req, res) => {
    res.status(404).json({
      success: false,
      error: "Route not found"
    });
  });
  // Centralized error handler
  app.use((err, req, res, next) => {
    console.error(err.stack);
    res.status(err.status || 500).json({
      success: false,
      error: err.message || "Internal Server Error"
    });
  });
  app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
  });

```

```
GET http://localhost:5000/api/tasks
```

```
Status: 200 OK
```

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "title": "Learn Node.js",
      "completed": false
    },
    {
      "id": 2,
      "title": "Learn Express.js",
      "completed": false
    },
    {
      "id": 3,
      "title": "Build REST API",
      "completed": false
    },
    {
      "id": 4,
      "title": "Learn React",
      "completed": false
    },
    {
      "id": 5,
      "title": "Test API with Postman",
      "completed": false
    }
  ]
}
```

Q1(c) Middleware

The logger is global, while authCheck is applied only to POST, PUT and DELETE routes. The complete implementation is included in server.js above.

```
[2026-09-23T09:10:12.120Z] GET /api/tasks
[2026-09-23T09:10:20.541Z] GET /api/tasks/1
[2026-09-23T09:10:35.002Z] POST /api/tasks
[2026-09-23T09:10:48.721Z] PUT /api/tasks/1
[2026-09-23T09:11:02.119Z] DELETE /api/tasks/1
```

Q2. Securing, Testing & Error Handling

Q2(a) CORS

```
// Enable CORS during development
app.use(cors());
// To temporarily disable CORS, comment the line above:
// // app.use(cors());
// After testing the browser failure, enable it again:
// app.use(cors());
```

When a browser client makes a cross-origin request while CORS is disabled, the browser blocks access to the response and reports a CORS policy error in the Developer Tools console. Re-enabling `app.use(cors())` allows the development client to access the API.

```
Browser Console:
Access to fetch at 'http://localhost:5000/api/tasks' from origin
'http://localhost:xxxx' has been blocked by CORS policy:
No 'Access-Control-Allow-Origin' header is present on the requested resource.
```

Q2(b) Postman / Thunder Client Requests

Collection: API Lab

1. GET http://localhost:5000/api/tasks

Header: none

2. GET http://localhost:5000/api/tasks/1

Header: none

3. POST http://localhost:5000/api/tasks

Header:

x-api-key: mysecretkey

Content-Type: application/json

Body:

```
{
  "title": "New Task",
  "completed": false
}
```

4. PUT http://localhost:5000/api/tasks/1

Header:

x-api-key: mysecretkey

Content-Type: application/json

Body:

```
{
  "title": "Updated Task",
  "completed": true
}
```

5. DELETE http://localhost:5000/api/tasks/1

Header:

x-api-key: mysecretkey

Test unauthorized POST:

Remove x-api-key.

Expected status: 401

POST http://localhost:5000/api/tasks

No x-api-key header

Status: 401 Unauthorized

```
{
  "success": false,
  "error": "Unauthorized: valid x-api-key is required"
}
```

Q2(c) 404 and Central Error Handler

Both handlers are already included at the end of server.js. The unmatched URL handler returns a JSON 404 response, while the four-parameter error middleware provides a consistent JSON error response.

GET http://localhost:5000/api/abc

Status: 404 Not Found

```
{
  "success": false,
  "error": "Route not found"
}
```

Q3. Consuming the API – Plain JavaScript Client

Q3(a–c) client/index.html

<!DOCTYPE html>

```

<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Task Manager - JavaScript Client</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      max-width: 700px;
      margin: 40px auto;
      padding: 20px;
    }
    li {
      margin: 10px 0;
    }
    #error {
      color: red;
    }
  </style>
</head>
<body>
  <h1>Task Manager</h1>
  <p id="loading">Loading...</p>
  <p id="error"></p>
  <ul id="taskList"></ul>
  <script src="script.js"></script>
</body>
</html>

```

Task Manager

- Learn Node.js - Pending
- Learn Express.js - Pending
- Build REST API - Pending
- Learn React - Pending
- Test API with Postman - Pending

Q3(a-c) client/script.js

```

const API_URL = "http://localhost:5000/api/tasks";
const loading = document.getElementById("loading");
const error = document.getElementById("error");
const taskList = document.getElementById("taskList");
async function loadTasks() {
  loading.textContent = "Loading...";
  error.textContent = "";
  taskList.innerHTML = "";
  try {
    const response = await fetch(API_URL);
    if (!response.ok) {
      throw new Error(`HTTP ${response.status}: ${response.statusText}`);
    }
    const result = await response.json();
    result.data.forEach(task => {
      const li = document.createElement("li");
      li.textContent =
        `${task.title} - ${task.completed ? "Completed" : "Pending"}`;
      taskList.appendChild(li);
    });
  } catch (err) {
    error.textContent =

```

```

    `Unable to load tasks. ${err.message}`;
  } finally {
    loading.textContent = "";
  }
}
loadTasks();

```

Failure-path test: Stop the Express server while index.html is open or refresh the page. The catch block displays a readable error instead of leaving the page blank.

Task Manager

Unable to load tasks. Failed to fetch

Result: When the server is stopped, the page remains usable and displays a clear error message instead of breaking.

Q4. Consuming the API – React Client

Q4(a) Create React App with Vite

```

cd api-lab
npm create vite@latest client-react -- --template react
cd client-react
npm install
npm install axios
npm run dev

```

Task Manager

Loading...

Q4(a-c) React TaskList – src/App.jsx

```

import { useEffect, useState } from "react";
import axios from "axios";
import "./App.css";
const API_URL =
  import.meta.env.VITE_API_URL || "http://localhost:5000";
const API_KEY = "mysecretkey";
function TaskList() {
  const [tasks, setTasks] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");
  const [title, setTitle] = useState("");
  async function fetchTasks() {
    try {
      setLoading(true);
      setError("");
      const response = await fetch(`${API_URL}/api/tasks`);
      if (!response.ok) {
        const data = await response.json().catch(() => ({}));
        throw new Error(data.error || `HTTP ${response.status}`);
      }
      const result = await response.json();
      setTasks(result.data);
    } catch (err) {
      setError(err.message || "Failed to load tasks");
    } finally {
      setLoading(false);
    }
  }
}

```

```

    }
    useEffect(() => {
      fetchTasks();
    }, []);
    // Axios version:
    // Axios parses JSON automatically, so response.json() is not required.
    // Axios rejects non-2xx responses automatically, so error handling uses catch.
    async function fetchTasksWithAxios() {
      try {
        setLoading(true);
        setError("");
        const response = await axios.get(`${API_URL}/api/tasks`);
        setTasks(response.data.data);
      } catch (err) {
        setError(
          err.response?.data?.error ||
          err.message ||
          "Failed to load tasks"
        );
      } finally {
        setLoading(false);
      }
    }
    async function addTask(e) {
      e.preventDefault();
      if (!title.trim()) {
        setError("Please enter a task title.");
        return;
      }
      try {
        setError("");
        const response = await fetch(`${API_URL}/api/tasks`, {
          method: "POST",
          headers: {
            "Content-Type": "application/json",
            "x-api-key": API_KEY
          },
          body: JSON.stringify({
            title: title.trim(),
            completed: false
          })
        });
        const result = await response.json();
        if (!response.ok) {
          throw new Error(result.error || `HTTP ${response.status}`);
        }
        // Update UI immediately without reload
        setTasks(prev => [...prev, result.data]);
        setTitle("");
      } catch (err) {
        setError(err.message || "Failed to create task");
      }
    }
    async function deleteTask(id) {
      try {
        setError("");
        const response = await fetch(`${API_URL}/api/tasks/${id}`, {
          method: "DELETE",
          headers: {
            "x-api-key": API_KEY
          }
        });
        const result = await response.json();
        if (!response.ok) {
          throw new Error(result.error || `HTTP ${response.status}`);
        }
      }
    }
  
```

```

    }
    // Remove task from UI immediately
    setTasks(prev => prev.filter(task => task.id !== id));
  } catch (err) {
    setError(err.message || "Failed to delete task");
  }
}
if (loading) {
  return <p>Loading...</p>;
}
return (
  <div className="container">
    <h1>Task Manager</h1>
    {error && <div className="error">{error}</div>}
    <form onSubmit={addTask}>
      <input
        value={title}
        onChange={e => setTitle(e.target.value)}
        placeholder="Enter task title"
      />
      <button type="submit">Add Task</button>
    </form>
    <button onClick={fetchTasksWithAxios}>
      Reload with Axios
    </button>
    {tasks.length === 0 ? (
      <p>No tasks found.</p>
    ) : (
      <ul>
        {tasks.map(task => (
          <li key={task.id}>
            <span>
              {task.title} -{" "}
              {task.completed ? "Completed" : "Pending"}
            </span>
            <button onClick={() => deleteTask(task.id)}>
              Delete
            </button>
          </li>
        ))}
      </ul>
    )}
  </div>
);
}
export default TaskList;

```

```

Task Manager

[ Enter task title ] [Add Task]

Learn Node.js - Pending [Delete]
Learn Express.js - Pending [Delete]
Build REST API - Pending [Delete]
Learn React - Pending [Delete]
Test API with Postman - Pending [Delete]

[Reload with Axios]

```

Q4(c) React CSS – src/App.css

```

.container {
  max-width: 700px;
  margin: 40px auto;
  font-family: Arial, sans-serif;
}
form {

```

```

display: flex;
gap: 10px;
margin-bottom: 20px;
}
input {
flex: 1;
padding: 10px;
}
button {
padding: 10px 14px;
cursor: pointer;
}
li {
display: flex;
justify-content: space-between;
margin: 10px 0;
}
.error {
color: #b00020;
background: #ffe5e5;
padding: 10px;
margin: 10px 0;
}

```

Before:

[Practice React] [Add Task]

After clicking Add Task:

Task Manager

```

Learn Node.js - Pending
Learn Express.js - Pending
Build REST API - Pending
Learn React - Pending
Test API with Postman - Pending
Practice React - Pending

```

The new task appears immediately without refreshing the page.

Q5. Full-Stack Error Handling, Config & Final Integration

Q5(a) 404 / 401 Error Handling

```

// Example client-side handling
const response = await fetch(`${API_URL}/api/tasks/9999`, {
  method: "DELETE",
  headers: {
    "x-api-key": API_KEY
  }
});
const result = await response.json();
if (!response.ok) {
  throw new Error(result.error || `HTTP ${response.status}`);
}

```

For a fake task id such as 9999, the server returns 404 with 'Task not found'. The React catch block displays that message in the UI rather than crashing or showing a blank screen. If the API key is missing or invalid, the server returns 401 and the same client-side error handling displays the readable message.

```
Action: Delete task with ID 9999
```

```
Server response:  
Status: 404 Not Found
```

```
{  
  "success": false,  
  "error": "Task not found"  
}
```

```
React UI:  
Task not found
```

```
Result:
```

Q5(b) Configuration

```
.env  
PORT=5000  
API_KEY=mysecretkey  
  
// client-react/.env  
VITE_API_URL=http://localhost:5000  
  
# server/.gitignore  
node_modules/  
.env  
# client-react/.gitignore should also ignore:  
.env  
.env.local
```

The server reads PORT and API_KEY from .env. The React application reads its API base URL from VITE_API_URL instead of hardcoding the localhost address in API calls.

```
server/.env  
  
PORT=5000  
API_KEY=mysecretkey  
  
client-react/.env  
  
VITE_API_URL=http://localhost:5000  
  
Result:  
Configuration values are read from environment files instead
```

Q5(c) Final Integrated Task Manager Mini App

Final project structure
api-lab/

```
server/  
  server.js  
  package.json  
  .env  
  .gitignore  
  
client/  
  index.html  
  script.js  
  
client-react/  
  .env  
  package.json  
  src/  
    App.jsx  
    App.css
```

How to Run the Complete Project

STEP 1: Start the backend

```
cd api-lab/server
```

```
npm install
```

```
npm run dev
```

Expected:

Server running at <http://localhost:5000>

STEP 2: Test the API

Open Postman or Thunder Client.

GET:

<http://localhost:5000/api/tasks>

POST:

<http://localhost:5000/api/tasks>

Header:

x-api-key: mysecretkey

Body:

```
{
  "title": "Practice React",
  "completed": false
}
```

STEP 3: Run the plain JavaScript client

Open client/index.html in a browser.

For best browser behavior, serve the client folder using a local static server.

STEP 4: Run React

```
cd api-lab/client-react
```

```
npm install
```

```
npm run dev
```

Open the Vite URL shown in the terminal.

STEP 5: Test errors

- Stop the backend and refresh the client: error message should appear.
- POST without x-api-key: server should return 401.
- DELETE task 9999: server should return 404 and React should display the error.
- Open an invalid API route: server should return 404 JSON.

Postman Collection Export – Optional Ready-to-Import JSON

```
{
  "info": {
    "name": "API Lab",
    "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json"
  },
  "item": [
    {
      "name": "GET All Tasks",
      "request": {
        "method": "GET",
        "url": "http://localhost:5000/api/tasks"
      }
    },
    {
      "name": "GET Task By ID",
      "request": {
        "method": "GET",
        "url": "http://localhost:5000/api/tasks/1"
      }
    },
    {
      "name": "POST Task",
      "request": {
        "method": "POST",
```

```

    "header": [
      {"key": "x-api-key", "value": "mysecretkey"},
      {"key": "Content-Type", "value": "application/json"}
    ],
    "body": {
      "mode": "raw",
      "raw": "{\"title\":\"Postman Task\",\"completed\":false}"
    },
    "url": "http://localhost:5000/api/tasks"
  },
  {
    "body": {
      "mode": "raw",
      "raw": "{\"title\":\"Updated From Postman\",\"completed\":true}"
    },
    "url": "http://localhost:5000/api/tasks/1"
  },
  {
    "name": "DELETE Task",
    "request": {
      "method": "DELETE",
      "header": [
        {"key": "x-api-key", "value": "mysecretkey"}
      ],
      "url": "http://localhost:5000/api/tasks/1"
    }
  }
]
}

```

Backend:

Express REST API + CRUD + Logger + API Key Auth
+ CORS + 404 Handler + Central Error Handler

Frontend:

React + Fetch + Axios + Create Task + Delete Task
+ Loading State + Error State